

Crashkurs



Dr. Sebastian Feld, Prof. Dr. Claudia Linnhoff-Popien
Steffen Illium, Jan Franz

Lehrstuhl für Mobile und Verteilte Systeme
Institut für Informatik
Ludwig-Maximilians-Universität München



- Kapitel 0: Python Geschichte
- Kapitel 1: Einrichtung
- Kapitel 2: Grundlagen
 - Syntax, Operatoren, Variablen,...
- Kapitel 3: Listen
 - Verwendung, Funktionen, Mutable vs. Immutable
- Kapitel 4: "Erste" Programmierschritte
 - Schleifen, Bedingungen, Fehlerhandling
- Kapitel 5: Ablaufsteuerung
 - If/else, Schleifentechniken



- Kapitel 6: Code-Style
- Kapitel 7: Weitere Datenstrukturen
 - Listen[2], Tupel, Sets, Dictionaries, Sequenz-Operatoren
- Kapitel 8: Module
 - Lebensdauer, Import, eigene Funktionen
- Kapitel 9: Beispiel: IoT-Verwaltung
 - Standardbibliotheken, Ausgabe-Formatierung, Klassen, I/O
- Kapitel 10: Fortgeschrittene Themen
 - Lambda Funktionen, Threading
- Kapitel 11: The Zen of Python

Kapitel 0

Python Basics

- Was ist Python?
- Python Geschichte

Stärken von Python



- Programmiersprache, entwickelt von Guido van Rossum
 - * 31. Januar 1956 in Haarlem, Niederlande
- Erste Version: 1991
 - Name stammt aus "Monty Python's Flying Circus"
- Aktuelle Version: 3.7.1 (20. Oktober 2018)
 - Inkompatibilität zu älteren 2.x Versionen
- Verfügbar auf Windows- und Unix-Systemen (Linux, Mac, ...)
- Anspruch: Förderung eines gut lesbaren und knappen Programmierstils
 - Oft als Skriptsprache verwendet
 - Bash-Code ist oft „Kraut und Rüben“ dagegen



- Einfach zu erlernen, aber mächtig
 - Viele wissenschaftliche Veröffentlichungen bieten zusätzlich entsprechende Bibliotheken
- Frei verfügbar, community driven
 - z.B. "Python Enhancement Proposals" (PEP) zur Errichtung von Best Practices
- Mehr "Funktionen" als Shell-/Batch-Skripte, z.B. Objekte
- Allgemeine Datentypen, flexibel einsetzbar
 - Multi-Purpose, fast alles was denkbar ist, ist machbar
- "Interpretierte" Sprache -> "kein" Kompilieren nötig
 - Java: Kompilierung in .class Dateien
 - Python: Implizite "Kompilierung" in Byte-Code, einfaches Ausführen der .py-Dateien genügt
 - Programmierung auf Windows-Maschine, Ausführung auf Linux-Server

Kapitel 1

Einrichtung

- Anaconda/Miniconda
- Paketverwaltungsprogramm „PIP installs packages“

Integrierte Entwicklungsumgebungen

- Code-Vervollständigung
- Automatische Formatierung
- Syntax-Highlighting

Freie und open-source Python Distribution

- Anaconda: inklusive Pakete und conda-GUI
- Miniconda: nur Repository Management

Ziel: Paket-Management vereinfachen

- Paket Manager conda kümmert sich um Abhängigkeiten
- Virtuelle Umgebungen

Eigenes Environment für jedes Projekt, um Ordnung zu halten

- Bsp: IoT-Environment ~180MB, Deeplearning/Keras-Environment ~1,5GB
- Env-Einstellungen exportieren, um auf anderer Maschine gleiche Gegebenheiten herzustellen

Best Practice: erst die Umgebung bereitstellen, dann programmieren



Miniconda:

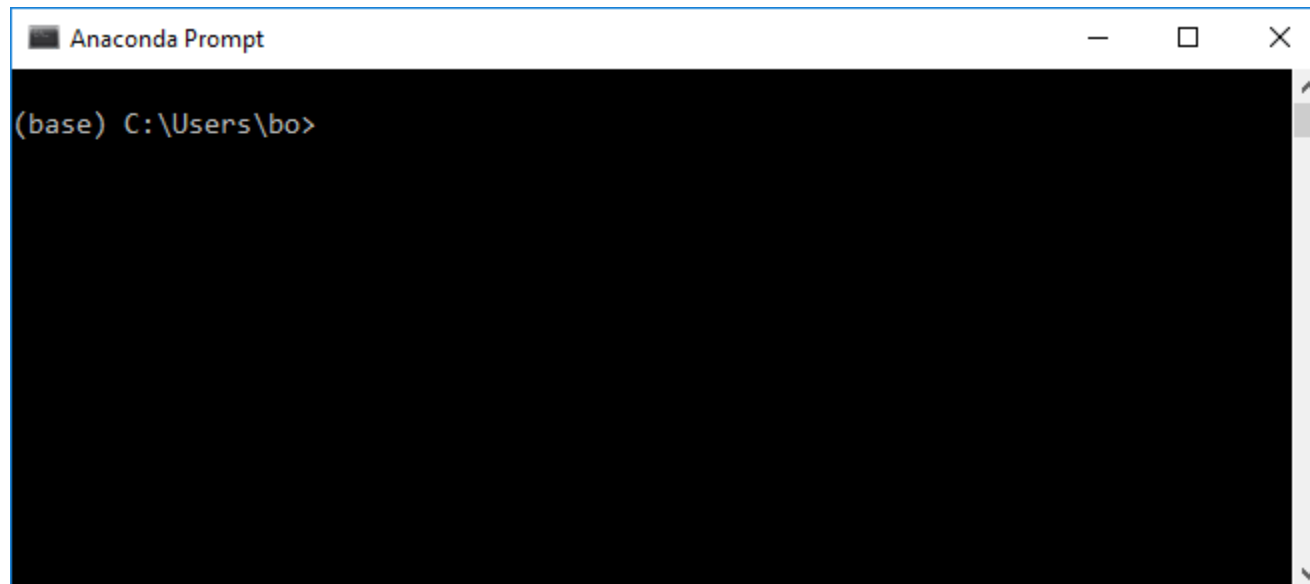
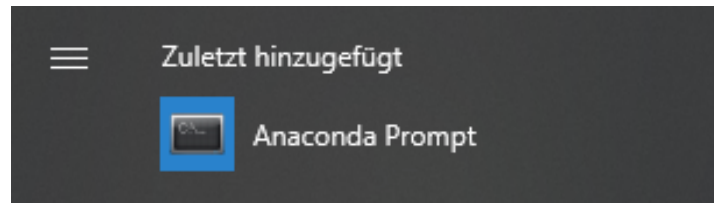
<https://conda.io/miniconda.html>

- Aktuelle Python Version: Python 3.7



Miniconda

	 Windows	 Mac OS X	 Linux
Python 3.7	64-bit (exe installer) 32-bit (exe installer)	64-bit (bash installer) 64-bit (.pkg installer)	64-bit (bash installer) 32-bit (bash installer)
Python 2.7	64-bit (exe installer) 32-bit (exe installer)	64-bit (bash installer) 64-bit (.pkg installer)	64-bit (bash installer) 32-bit (bash installer)





```
$ conda create -n iot python=3.6
```

```
# Linux\Mac:
```

```
$ conda activate iot
```

```
# Windows:
```

```
$ activate iot
```

```
$ python
```

```
Anaconda Prompt

(base) C:\Users\bo>conda create -n iot python=3.6
Solving environment: done

## Package Plan ##

  environment location: C:\Users\bo\Miniconda3\envs\iot

  added / updated specs:
    - python=3.6

The following packages will be downloaded:

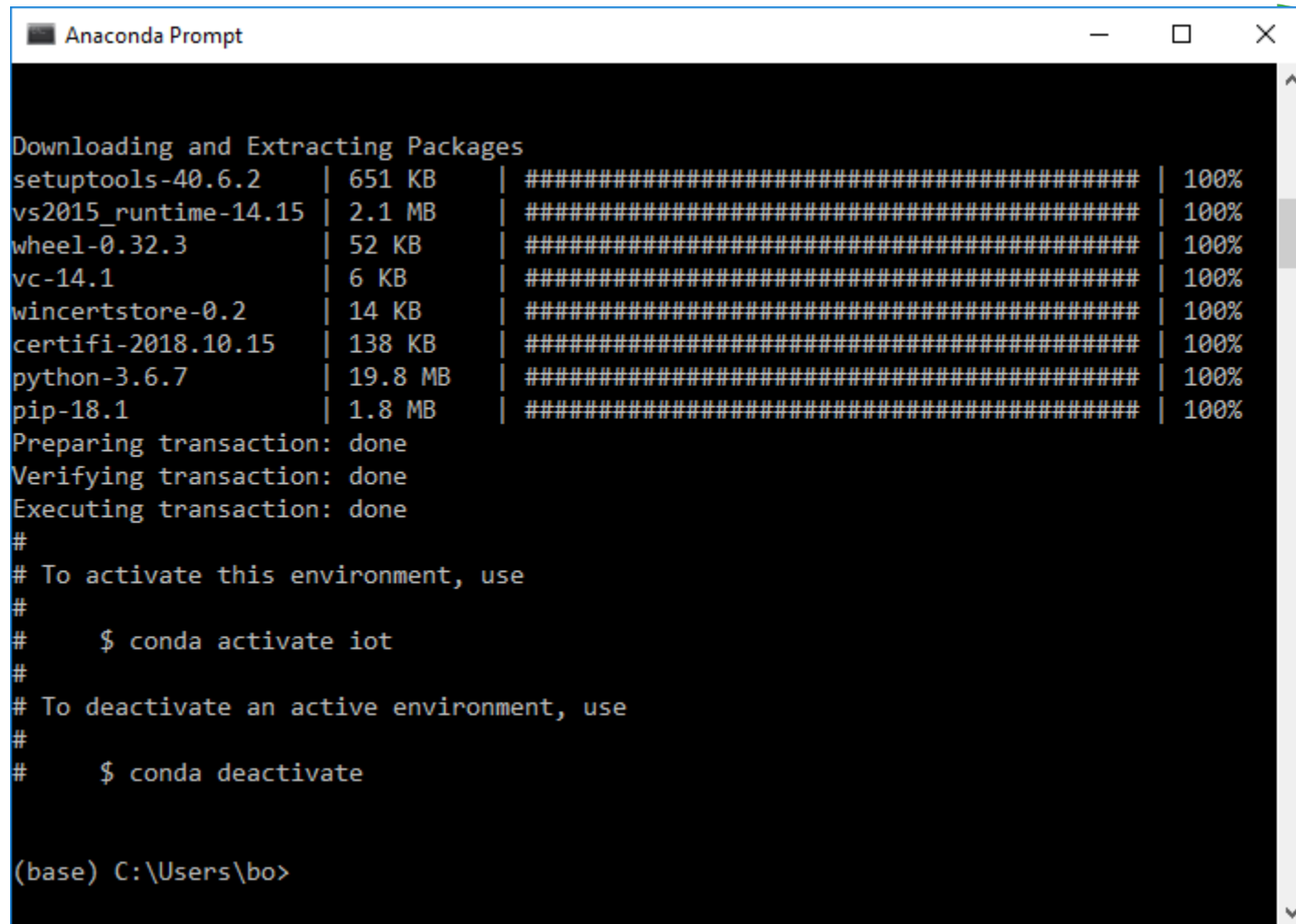
package                        |          build          |
-----|-----|
setuptools-40.6.2              |      py36_0             | 651 KB
vs2015_runtime-14.15.26706    |    h3a45250_0           | 2.1 MB
wheel-0.32.3                   |      py36_0             | 52 KB
vc-14.1                         |    h0510ff6_4           | 6 KB
wincertstore-0.2               |   py36hcdd9a18_0        | 14 KB
certifi-2018.10.15             |      py36_0             | 138 KB
python-3.6.7                   |    h33f27b4_1           | 19.8 MB
pip-18.1                       |      py36_0             | 1.8 MB
-----|-----|
Total:                         |                        | 24.5 MB

The following NEW packages will be INSTALLED:

certifi:      2018.10.15-py36_0
pip:          18.1-py36_0
python:       3.6.7-h33f27b4_1
setuptools:   40.6.2-py36_0
vc:           14.1-h0510ff6_4
vs2015_runtime: 14.15.26706-h3a45250_0
wheel:        0.32.3-py36_0
wincertstore: 0.2-py36hcdd9a18_0

Proceed ([y]/n)?
```





Anaconda Prompt

```
Downloading and Extracting Packages
setuptools-40.6.2 | 651 KB | ##### | 100%
vs2015_runtime-14.15 | 2.1 MB | ##### | 100%
wheel-0.32.3 | 52 KB | ##### | 100%
vc-14.1 | 6 KB | ##### | 100%
wincertstore-0.2 | 14 KB | ##### | 100%
certifi-2018.10.15 | 138 KB | ##### | 100%
python-3.6.7 | 19.8 MB | ##### | 100%
pip-18.1 | 1.8 MB | ##### | 100%
Preparing transaction: done
Verifying transaction: done
Executing transaction: done
#
# To activate this environment, use
#
#     $ conda activate iot
#
# To deactivate an active environment, use
#
#     $ conda deactivate

(base) C:\Users\bo>
```



```
Anaconda Prompt - python

(base) C:\Users\bo>activate iot

(iot) C:\Users\bo>python
Python 3.6.7 |Anaconda, Inc.| (default, Oct 28 2018, 20:57:12) [MSC v.1915 32 bit (Intel)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

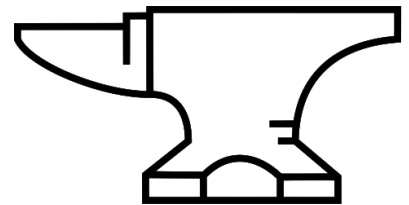
Anaconda beinhaltet MKL-Optimierung

- “Developed specifically for science, engineering, and financial computations, Intel™ Math Kernel Library (MKL) is a set of threaded and vectorized math routines that work to accelerate various math functions and applications. Anaconda has packaged MKL-powered binary versions of some of the most popular numerical/scientific Python libraries into MKL Optimizations for improved performance.”
- <https://docs.anaconda.com/mkl-optimizations/>



Zusätzlicher Paket channel: Conda-Forge

- “A community led collection of recipes, build infrastructure and distributions for the conda package manager.”
- Sehr gut, aber langsam (Latenz der Forge-Server).
- <https://conda-forge.org/>



CONDA-FORGE

Es ist möglich, conda- und pip-Pakete zu mischen

- Installationen mittels pip innerhalb von conda Umgebung
- Best Practice: besser nicht mischen, außer man hat keine andere Wahl
- Conda kann pip-Pakete nicht verwalten, evtl. doppelte Bibliotheken



Verbreitete Pakete sind: Numpy, scipy, matplotlib, jupyter, pandas

```
# activate environment
```

```
$conda activate iot
```

```
# install pip package
```

```
$ pip install <package-name>
```

```
# see all installed packages (current env)
```

```
$ conda list
```

Mehrere IDEs zur Entwicklung verfügbar:

Spyder

<https://anaconda.org/anaconda/spyder>

Editor (Notepad++)

<https://notepad-plus-plus.org/>

IntelliJ PyCharm

<https://www.jetbrains.com/pycharm/>





```
20 """
21 response = self.client.get(reverse('polls:index'))
22 self.assertEqual(response.status_code, 200)
23 self.assertContains(response, "No polls are available.")
24 self.assertQuerysetEqual(response.context['latest_question_list'], [])
25 self.test
26
27 def test_index_view_with_a_future_question(self):
28     """
29     Questions with a pub_date in the future should not be displayed on
30     the index page.
31     """
32     create_question(question_text="Future question.", days=30)
33     response = self.client.get(reverse('polls:index'))
34     self.assertContains(response, "No polls are available.",
35                         status_code=200)
36     self.assertQuerysetEqual(response.context['latest_question_list'], [])
37
38 def test_index_view_with_future_question_and_past_question(self):
39     """
40     Even if both past and future questions exist, only past questions
41     should be displayed.
42     """
43     create_question(question_text="Past question.", days=-30)
44     create_question(question_text="Future question.", days=30)
45     response = self.client.get(reverse('polls:index'))
46     self.assertQuerysetEqual(
47         response.context['latest_question_list'],
48         ['<Question: Past question.>']
49     )
50
51 def test_index_view_with_two_past_questions(self):
52     """
53     """
54     create_question(question_text="Past question.", days=-30)
55     create_question(question_text="Past question.", days=-30)
56     response = self.client.get(reverse('polls:index'))
57     self.assertQuerysetEqual(
58         response.context['latest_question_list'],
59         ['<Question: Past question.>', '<Question: Past question.>']
60     )
```

Statement seems to have no effect. Unresolved attribute reference 'test' for class 'QuestionViewTests'.

Kapitel 2

Grundlagen

- Verwendung der Konsole
- Mathematische Operatoren

Variablen

- Mehrfachzuweisungen
- Kurzschreibweise

Ausgabe

- `print()`

Zeichenketten

- Indizes
- `len()`



- Python beherrscht alle Grundrechenarten (und mehr)
 - $+$, $-$, $*$, $/$
 - Kommazahlen mit "." als Separator angeben, sonst erfolgt automatische Umwandlung in "Sammlungen"

- Ausgabe direkt in der nächsten Zeile:

```
>>> 10367 * 42 / 5  
87082.8
```

- "Punkt-Vor-Strich" wird beachtet:

```
>>> 10 + 10 * 2  
30
```

- Ganzzahlige Werte werden als Integer bezeichnet



- Klammern:

- Runde Klammern "(" und ")" zur Kontrolle über die Reihenfolge

```
>>> (10+10) * 2
40
```

```
>>> (10 + (10 * 3)) * 2
80
```

- Eckige "[" "]" / Geschwungene "{" "}" haben gesonderte Funktionalität

- Division "/" vs. Ganzzahldivision "//":

- Praktisch, spart ein Casting von Float zu Integer, falls so benötigt

- Rest (Modulo): "%"

```
>>> 7 % 3
1
```

```
>>> 7 / 3
2.3333333333333335
>>> 7 // 3
2
```



- (Vor Python 3) zwei Kategorien: „Signed Integers“ und „Long Integers“
- Signed Integers
 - Ganzzahlige Zahlen, positiv oder negativ
 - 32 Bit
 - ⇒ 2.147.483.647 als maximale Zahl
 - +1 Addition auf diese Zahl führte zu -2.147.483.648
- Long Integers
 - Ganzzahlige Zahlen, positiv oder negativ
 - Prinzipiell unbeschränkt in der Größe
 - Schreibweise: `myLongNumber = 982743641L`



- Seit Python 3: Nur noch Long Integers
- Long Integers
 - Ganzzahlige Zahlen, positiv oder negativ
 - Prinzipiell unbeschränkt in der Größe
 - Schreibweise: `myLongNumber = 982743641`
- Einzig beschränkt in der Zeichenanzahl
 - Abhängig von der maximalen Wortgröße des Systems
 - Beispiel Windows: 32 Bit (Standard)
 - Statt 2.147.483.647 als maximale Zahl nun also eine Zahl mit 2.147.483.647 **Ziffern**

Abstract

This document describes the development and release schedule for Python 2.7.

Update

The End Of Life date (EOL, sunset date) for Python 2.7 has been moved five years into the future, to 2020. This decision was made to clarify the status of Python 2.7 and relieve worries for those users who cannot yet migrate to Python 3. See also [PEP 466](#).

This declaration does not guarantee that bugfix releases will be made on a regular basis, but it should enable volunteers who want to contribute bugfixes for Python 2.7 and it should satisfy vendors who still have to support Python 2 for years to come.

There will be no Python 2.8 (see [PEP 404](#)).



- Fließkomma-Zahlen
 - `myFloat = .1` (für 0.1) Oft verwendet, aber Erhöhung der Lesbarkeit?
- Darstellung in Binärzahl - Kombination aus Brüchen mit der Basis 2
 - Dezimal $0.125 = 1/10 + 2/100 + 5/1000$
 - Binär: $0.001 = (0/1 +) 0/2 + 0/4 + 1/8$
- Brüche sind nur bedingt genau (wie überall auch)
 - Nicht jede Zahl als Bruch zur Basis 2 darstellbar
 - Nicht unendlich viele Bits zum Darstellen
- IEEE 754-2008 definiert Standards: 16, 32, 64 und 128 Bit
- Selbst in aktueller Version also ungenaue Ergebnisse möglich
 - Unumgänglich, aber zu beachten!



- Python versteht auch komplexe Zahlen
 - Verwendet in verschiedenen Naturwissenschaften
 - Beispiel: Berechnung von Wechselströmen in der Elektrotechnik

- Kennzeichnung durch Suffix "i" oder "j",

```
>>> 1j + 4j  
5j
```

```
>>> (3 + 1j) * 3  
(9+3j)
```

- Trennung immer in zwei Teile: Realteil und imaginärer Teil



- Bisher: Keine Verwendung der Ausgabe
 - Interpreter "vergisst" die Zahl einfach

- "Variablen" als Zwischenspeicherung
 - Beispiel: `breite = 5`

⇒ Eingabe `breite` führt zur Ausgabe 5

- Variablen
 - Müssen immer mit einem Buchstaben beginnen
 - Sind Case-Sensitive (Groß-/Kleinschreibung beachten!)
- Erneutes Belegen der Variable überschreibt den Wert (unwiderruflich!)

```
>>> breite = 5
>>> breite = 3
>>> breite
3
```



```
breite = 5  
hoehe = 3
```

- Mathematische Operationen sind möglich:

- `breite * hoehe` gibt 15 zurück
- `flaeche = breite * hoehe` speichert diesen Wert in der Variablen `flaeche`

```
>>> breite = 5  
>>> hoehe = 3  
>>> flaeche = hoehe * breite  
>>> flaeche  
15
```



- "Speicherung" (in aller Regel) nur der aktuelle Wert
- Veränderung an "breite":

```
>>> breite = 10 (Die Fläche müsste also jetzt 30 sein)
>>> flaeche
15
```

```
>>> flaeche = breite * hoehe
```

 würde den Wert aktualisieren



Speicherung in mehrere Variablen

```
>>> hoehe = breite = 5
```

⇒ Beide Variablen erhalten den Wert 5

```
>>> hoehe, breite, tiefe = 5, 4, 3
```

entspricht

```
>>> hoehe = 5  
>>> breite = 4  
>>> tiefe = 3
```

„Entpacken“ fördert nicht zwingend die Lesbarkeit, reduziert jedoch den Zeilenverbrauch



Potentieller Logikfehler:

Ungleiche Anzahl auf beiden Seiten:

```
>>> hoehe, breite, tiefe = 5, 4
```

```
Traceback (most recent call last):
```

```
  File "<stdin>", line 1, im <module>
```

```
ValueError: not enough values to unpack (expected 3, got 2)
```



```
>>> hoehe = 5
```

Veränderung einer Zahl:

```
>>> hoehe = hoehe + 2
```

Gleiche Operation, syntaktisch kürzer via Zuweisungsoperator:

```
>>> hoehe += 2
```

Funktioniert entsprechend auch mit `-=`, `*=` und `/=`



- Variablen müssen vor Verwendung definiert sein

```
>>> a = 5
```

- Definition geschieht durch Zuweisung, bspw.:

```
>>> b
```

- Die Eingabe
würde also folgenden Fehler verursachen:

```
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
NameError: name 'b' is not defined
```



- Nicht jede Eingabe soll vom Interpreter gedeutet werden
- "#" gibt den Beginn einer Kommentarzeile an

```
>>> a = 5 # Dies ist ein sinnvoller Kommentar
```

- Variable hat den Wert 5 nach Ausführung
- Kommentar wird ignoriert (und hierbei also verworfen)
- Kommentare (laut Standard)
 - Dienen ausschließlich zur Erläuterung des Codes für den Lesenden
 - Sollten immer komplette Sätze sein
 - Dürfen der Code-Funktionalität nicht widersprechen
 - Prinzipiell in Englisch

- Eingabe

```
>>># Kommentar ohne Code
```

- Reaktion Interpreter:

```
>>># Kommentar ohne Code  
...
```

- Eingabe wird durch Kommentar nicht beendet
- Interpreter wartet auf weitere Eingabe
- Beenden durch Befehl + Enter
 - z.B.: `myNumber = 10`
 - Auch leere Anweisung möglich
- In Skripten sollte ein Kommentar über Befehl oder Condition stehen



- Ausgegebene Ergebnisse sind doch nicht verloren ;-)
- Zuletzt ausgegebener Ausdruck gespeichert in _-Variable

```
>>> 100 * 3.141
314.1
```

```
>>> tmp = _
>>> tmp
314.1
```

```
>>> tmp = _
>>> tmp = _
>>> tmp
314.1
```



- Python kann mit mehr als nur Zahlen umgehen
- Speichern von Zeichenketten (sogenannte "Strings") in Variablen

```
>>> tmp = some text
```

Funktioniert nicht, da der Interpreter versucht, die Variablen "some" und "text" aufzulösen

```
>>> tmp = "some text"
```

Funktioniert auch mit einfachen Hochkommata:

```
>>> tmp = 'some text'
```

Technisch gesehen das gleich, jedoch sind Hochkommata zu bevorzugen

String mit Zitat:

```
>>> tmp = 'Parameter "p" ist ein Integer.'
```


- Ausgabe von "tmp" gibt wie bei Zahlen den Wert aus

```
>>>tmp  
"some text"
```

- Allerdings umgeben von Anführungszeichen
 - Python bietet Funktionen zur sauberen Ausgabe an
- Funktionen arbeiten mit vorhandenen Werten
 - Können beliebig viele (auch keine) Werte als Argumente erwarten
 - Syntax: `FUNKTION (ARGUMENT, ARGUMENT, ...)`
- Beispiel: `print(VARIABLE)`

```
>>> print(tmp)  
some text
```



- Mehrzeilige Speicherung von Strings
- Enter-Taste führt zum Absenden des Befehles
 - Shift+Enter funktioniert auch nicht

```
SyntaxError: EOL while scanning string literal
```

- Nutzung von Sonderzeichen:

```
>>> tmp = "some \n text"
```

- Normale Ausgabe ignoriert die Interpretation der Sonderzeichen:

```
>>> tmp  
"some \n text"
```



- Mithilfe des `print()` Befehls:

```
>>> print(tmp)
some
text
```

- Zu beachten: Das Leerzeichen vor "text"

```
>>> tmp = "some \ntext"
```

- Da nur ein Zeichen von dem Backslash gedeutet wird, funktioniert der obige Befehl

```
>>> print(tmp)
some
text
```



- Es existieren einfache Operatoren für Strings
 - Ähnlich wie mathematische Operatoren? ☐ Yes...
- "+" verknüpft zwei Strings / Stringvariablen

```
>>> stringA = "eine"  
>>> stringB = "Nachricht"  
  
>>> stringAB = stringA + stringB
```



```
>>> stringAB  
"eineNachricht"
```

- Beliebige Mischung von Variablen und Strings möglich

```
>>> stringAB = stringA + " " + stringB  
>>> stringAB  
"eine Nachricht"
```

- Wichtig hierbei: Beide Variablen müssen Strings enthalten
- Korrekt, aber besser: Verwendung von „Text Formatting“

```
>>> '{0} {1}'.format(stringA, stringB)  
"eine Nachricht"
```



- Vor Python 3: Unicode als Standard
- Einheitlicher und internationaler Standard für alle "sinntragenden" Schriftzeichen
 - Erste Version 1991
- Wird von Python unterstützt
 - ⇒ Sonderzeichen können – gesondert – verwendet werden
- z.B. hat α hat den Unicode-Wert U+03B1

```
>>> stringA = "Ein alpha \u03B1"
```

```
>>> stringA = "Ein alpha \u03B1"  
>>> stringA  
"Ein alpha  $\alpha$ "
```

```
>>> print(stringA)  
Ein alpha  $\alpha$ 
```



- Heutiger Standard: UTF-8
- Auf jedem System unterstützt
- Wird von Python standardmäßig verwendet
 - ⇒ Sonderzeichen können verwendet werden
 - ⇒ Ohne gesonderte Markierung

```
>>> stringA = "ein Umlaut Ö"
```

- Ausgabe

```
>>> print(stringA)  
ein Umlaut Ö
```



- Einzelne Zeichenpositionen aus Strings sind auslesbar
- Zählung startet bei 0
- Zugriff über eckige Klammern

- VARIABLE[POSITION]

```
>>> stringA = "hallo"  
>>> stringA[1]  
'a'  
>>> print(stringA[1])  
a
```

- Negativer Index zählt von hinten:

```
>>> print(stringA[-1])  
o
```


- Das letzte Zeichen eines Strings mit unbekannter Länge herausgeben?
- `len(VARIABLE)` gibt die Länge des angegebenen Strings aus

```
>>>len(stringA)
```

```
5
```

```
>>>stringA[len(stringA)]
```

```
IndexError: string index out of range
```

- Fehlerquelle! Anzahl der Zeichen ist NICHT null-basiert

```
>>>stringA[len(stringA)-1]
```

```
'o'
```

Kapitel 3

Datentyp: Listen

- Indizes, len(), etc
- Slicing
- Verschachtelung

Mutable vs. Immutable



- Bisher: stringA, stringB, stringC, als "Sammlung"
- Python bietet aber auch eigene Typen dafür
- z.B. "list":
 - Liste von Werten
 - Sehr flexibel
 - Gekennzeichnet durch eckige Klammern und Kommata

```
>>> myList = [0, 1, 2, 3, 4, 5]
>>> myOtherList = ["Milch", "Brot", "Aufschnitt"]
>>> myEmptyList = []
```

- Code-Style: List-Elemente (möglichst) einheitlich halten
- Schlechtes Beispiel:

```
>>> badList = [1, "2", "Brot", 3, "fünf"]
```



- Verwendung ähnlich wie Strings:

```
>>> myOtherList = ["Milch", "Brot", "Aufschnitt"]
```

```
>>> myOtherList  
["Milch", "Brot", "Aufschnitt"]
```

```
>>> myOtherList[0]  
"Milch"
```

```
>>> len(myOtherList)  
3
```

```
>>> myOtherList + myOtherList  
["Milch", "Brot", "Aufschnitt", "Milch", "Brot", "Aufschnitt"]
```



- **Mathematische Operatoren:**
 - Lediglich „vergrößernde“ Ausdrücke, wie „+“ und „*“

```
>>> myList = ["Milch", "Brot", "Aufschnitt"]
>>> myOtherList = ["Zucker", "Mehl"]

>>> myList += myOtherList
>>> myList
['Milch', 'Brot', 'Aufschnitt', 'Zucker', 'Mehl']

>>> myList = ["Milch", "Brot", "Aufschnitt"]
>>> myList *= 2
>>> myList
['Milch', 'Brot', 'Aufschnitt', 'Milch', 'Brot', 'Aufschnitt']
```



- Listen können auch nur teilweise ausgegeben/verwendet werden
- Sogenanntes Slicing: „In Scheiben schneiden“

```
>>> myOtherList[2:]
```

Alle Elemente ab dritten Eintrag (inklusive)

```
>>> myOtherList[:2]
```

Alle Elemente bis zum dritten Eintrag (exklusive)

Negative Indizes beginnen am Ende der Liste

- Alle Elemente ausschließlich des ersten und letzten Eintrags

```
>>> myOtherList[1:-1]
```

Alle Elemente

```
>>> myOtherList[:]
```

- Slicing funktioniert genauso für Zeichenketten

```
>>> myString[2:]
```

Alle Zeichen ab der dritten Position (inklusive)

```
>>> myString[:2]
```

Alle Zeichen bis zur dritten Position (exklusive)

Negative Indizes beginnen am Ende der Liste

- Alle Zeichen ausschließlich der ersten und letzten Position

```
>>> myString[1:-1]
```

Alle Zeichen

```
>>> myString[:]
```



- Listen sind im Gegensatz zu Variablen auch stückweise veränderbar ("mutable")
- Veränderung einzelner Werte ohne komplettes Überschreiben der Liste möglich
 - Einfache Variablen „nur“ komplett überschreibbar („immutable“)

```
>>> myList = ['eine', 'lange', 'Liste']  
>>> myList  
['eine', 'lange', 'Liste']  
  
>>> myList[1] = "kurze"  
>>> myList  
['eine', 'kurze', 'Liste']
```




- Alles in Python ist ein Objekt
 - Jedes Objekt hat eine ID als Verweis
 - Jede Variable "hält" ein Objekt als Wert
- Strings sind nicht veränderbar => immutable
 - `myNumber = 5` erstellt ein Objekt und speichert den Verweis in `myNumber`
 - `myNumber = myNumber + 1` erstellt wieder ein neues Objekt mitsamt Verweis
- Listen hingegen sind veränderbar => mutable
 - Listen sind eine Sammlung solcher Verweise
 - Veränderung eines Elements: Erstellung eines neuen Objektes mit Austausch des Verweises
- Grobe Regel:
 - Einfache Datentypen sind tendenziell immutable
 - Container-ähnliche Datentypen sind tendenziell mutable

Wird uns später noch
einmal begegnen
(`del()`)



- Alles in Python ist ein Objekt

```
>>> A=B=5
>>> A = A+ 1
>>> id(A), id(B)
(1682426064, 1682426048)
```

```
>>> A=B=list()
>>> A.append(4)
>>> Id(A), ID(B)
(19006168, 19006168)
```



- Auch Variablen können in die Liste geschrieben werden

```
>>> a,b,c = "IoT", "ist", "toll"  
>>> myList = [a,b,c]  
>>> myList  
['IoT', 'ist', 'toll']
```

- Auch hier gilt: Es wird der Wert der Variablen bei der Zuweisung verwendet

```
>>> c = "okay"  
>>> myList  
['IoT', 'ist', 'toll']
```

⇒ Verweis der Liste wurde nicht geändert

- Nützlich unter anderem bei Speicherung von Zwischenergebnissen



- Wir haben bislang...

- Werte in Listen
- Variablen in Listen

- Listen in Listen?

```
>>> listA = [1,2,3]
>>> listB = [4,5,6]
>>> listC = [7,8,9]
>>> listList = [listA, listB, listC]

>>> listList
[[1, 2, 3], [4, 5, 6], [7, 8, 9]]
>>> listList[0]
[1, 2, 3]
>>> listList[0][1]
2
```

- Verschachtelung mit allen Objekt-Typen und Tiefen möglich (Liste in Liste in Liste...)
- Best Practice: Bis Tiefe 2, ab dann spezielle Konstrukte verwenden

Kapitel 4

Bisher lediglich einfache Befehle

- Mehr oder minder hardcodiert

Schleife: while

Bedingungen

- Datentyp: Boolean

Verwenden von Benutzereingaben

Fehlerhandling



- Schleife zur wiederholten Ausführung von Befehlen
- Ermöglicht komplexere Programme
- Aufbau:

```
>>> while BEDINGUNG:  
...     code #Einrückung beachten  
...
```



- Bedingungen dienen zur Kontrolle von Ausführungen
- Geben `True` oder `False` zurück
 - "Boolean" Datentyp kann nur diese beiden Werte ergeben
- Einfache Bedingungen:
 - `a < b` #a kleiner als b
 - Weitere Operatoren:
 - `<=` (kleiner oder gleich)
 - `>` (größer)
 - `>=` (größer oder gleich)
 - `==` (gleich)
 - `!=` (ungleich)



- Eindeutig bei Zahlen:

```
3 < 5 #a kleiner als b
```

- Bei Zeichenketten?

```
'apple' < 'banana'
```

- Zeichenweise von links nach rechts
- Hinter jedem Zeichen steckt ein Zahlenwert
 - Definiert über ASCII: <https://en.wikipedia.org/wiki/ASCII>
 - 'A' (65) ist nicht gleich 'a' (97)
- Wenn gleich, dann nächste beiden Zeichen vergleichen...



- Eindeutig bei Zahlen:

```
3 < 5 #a kleiner als b
```

- Bei Listen?

```
[1, 2, 3, 4, 5] < [9, 8, 7, 6, 5]
```

- Auch hier: Elementweise von links nach rechts
 - Pro Element wieder üblicher Vergleich
 - Wenn gleich, dann nächste beiden Zeichen vergleichen
- Wenn gleich, dann nächste beiden Elemente vergleichen...



- Python benutzt **keine** Klammern zum Umfassen von Code-Blocks

- Beispiel: Schleife in Java:

```
while (Bedingung) {  
    // Code  
}
```

Java

- Stattdessen: Gruppierung durch Einrücken

- Tabs oder Leerzeichen
 - Gleiche Blöcke müssen gleich eingerückt sein
 - Standard: 4 Leerzeichen, keine Tabs
 - IDEs hilfreich durch automatische Unterstützung

- **while** BEDINGUNG:
 code #Einrückung beachten

Python

- Wenn zu viele Einrückungen, dann ggfs. in Funktionen auslagern



- Beispiel:

```
>>> a = 0
>>> while a < 5:
...     print(a)
...#nochmal Enter drücken
```

Ausgabe:

0
0
0

etc..

Endlosschleife, STRG+C zum Abbrechen



- Beispiel:

```
a = 0
while a < 5:
    a = a + 1
    print(a)
#nochmal Enter drücken
```

Ausgabe:

```
1
2
3
4
5
```

5 wird ausgegeben, weil **erst** geprüft, **dann** erhöht und **dann** ausgegeben wird!



- Eigene Multiplikation:

```
faktor = 3
ende = 4
counter = result = 0

while counter < ende:
    result += faktor
    counter += 1

print(result)
```

Ausgabe:

12

„Augmented Assignment“ (+=) ist manchmal nicht gut zu lesen

- https://docs.python.org/3/reference/simple_stmts.html#augmented-assignment-statements



- Verknüpfung von Bedingungen

- Logisches "ODER "

- `True` wenn mindestens eine Bedingung zutrifft

	True	False
True	True	True
False	True	False

```
a < b or a < c #a kleiner als b ODER a kleiner als c
```

- Logisches "UND"

- `True` nur dann, wenn beide Bedingungen zutreffen

	True	False
True	True	False
False	False	False

```
a < b and a < c #a kleiner als b UND a kleiner als c
```

- Schönes Beispiel für die Lesbarkeit von Python durch Benutzung „menschlicher“ Sprache



- Auch hier wieder beliebige Schachtelung möglich
- Einfaches, statisches Beispiel: `True and (True or False)`
 - Ergibt `True`
- Beispiel 2: `(True or (False or False)) and (True or False)`
 - ?
- Schon korrekt, aber Lesbarkeit?

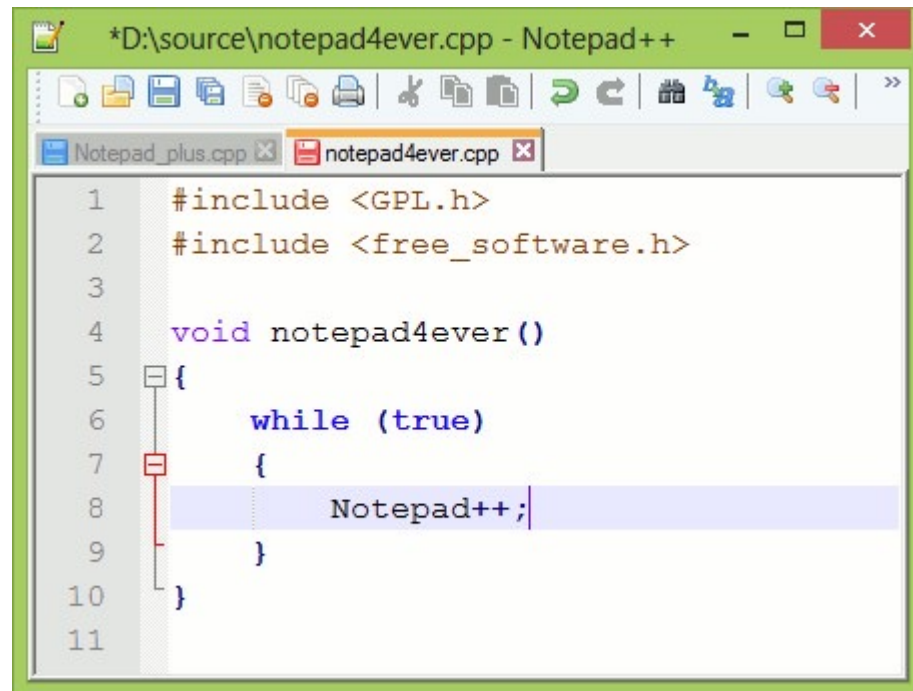


- Boolean basiert auf Integer-Klasse
 - `False` entspricht `0`
 - `True` entspricht `1`
- Auswertung mit `bool (PARAMETER)`
 - Funktioniert im Allgemeinen mit allen Objekten
- Generell: Eingabe ergibt `True`, außer die Eingabe ist...
 - Eine Bedingung, die `False` ergibt
 - `None` oder `False`
 - `0` in jedem Zahlentyp (`0`, `0.0`, `0j`, ...)
 - Leeres Objekt wie `myEmptyString = ""`
- Praktisch, wenn man testen möchte, ob bspw. ein Parameter gesetzt wurde
 - `if variable`

- Steigende Anforderungen erfordern größere Skripte
- Nur schwer bzw. gar nicht über Konsole lösbar

⇒ Verwendung von externen Editoren zur Vorbereitung

- Notepad++
- ...
- Dateiformat: .py



```
1  #include <GPL.h>
2  #include <free_software.h>
3
4  void notepad4ever()
5  {
6      while (true)
7      {
8          Notepad++;
9      }
10 }
11
```



- Zum Ausführen:
 - Speichern nicht vergessen...
 - Interpreter Modus verlassen (`quit()`)
 - (Zur Datei navigieren)
 - Linux: `python DATEINAME`

Ausführung bringt gewohnte Ausgabe:

```
a = 0
while a < 5:
    print(a)
    a = a + 1
```

Best Practice: Dateinamen so kurz und einfach wie möglichen halten

- Programmabläufe nach wie vor statisch
- Eingabemöglichkeit von Werten: "input(OPTIONAL_MESSAGE)"
- Beispiel:

```
name = input("Bitte Namen eingeben: ")  
print("Hallo " + name)
```

- Programm wartet mit der restlichen Durchführung, bis ein Wert eingetragen und mit Enter bestätigt wurde
- Im Beispiel wird print() also erst nach Eingabe des Namens aufgerufen
- Eher zum Zwecke einer Demo
 - Python-Skripte sollten eher „vorkonfiguriert“ werden
 - input() blockiert den Main-Thread



Python-Fehlermeldungen:

- Werden bei Tippfehlern, Logikfehlern, etc. geworfen
- (Meistens) hilfreich
- z.B. Division durch 0:

```
>>> 10 / 0
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ZeroDivisionError: division by zero
```

- Zu sehen: Fehler in Zeile 1 (wir haben ja nur eine)
- Fehlerursache: "ZeroDivisionError: division by zero"

- Exponent-Rechner:

```
base = input("Bitte Basis eingeben: ")
exponent = input("Bitte positiven Exponenten eingeben: ")
counter = 1
result = base
while counter < exponent:
    result = result * base
    counter = counter + 1
print(result)
```

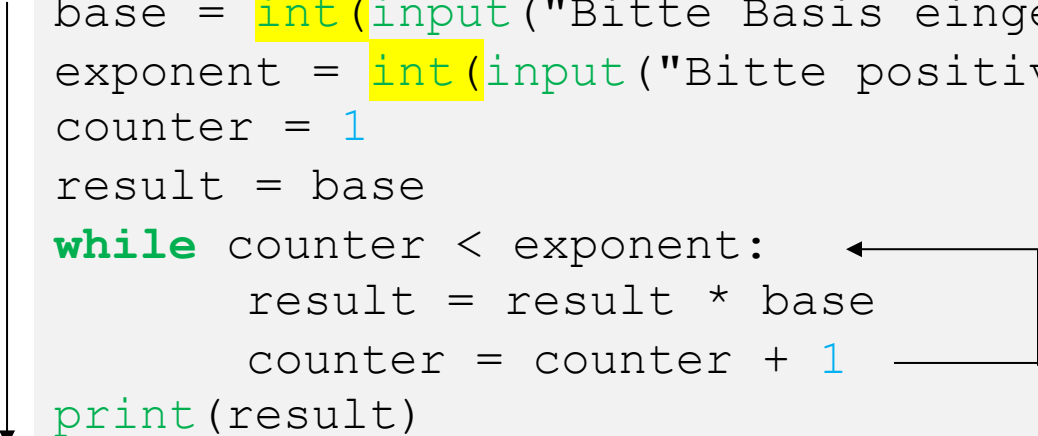
- Ausführung des Taschenrechners:

```
Traceback (most recent call last):  
  File "C:\test.py", line 5, in <module>  
    while counter<exponent:  
TypeError: '<' not supported between instances of 'int' and 'str'
```

- **input()** gibt standardmäßig einen String zurück
 - Operand "<" zwischen Zahl (counter) und Zeichenkette (exponent) nicht anwendbar
- Umwandlung: **TYP**(WERT)
- Hier also:
 - base = **int**(**input**("Bitte positiven Exponenten eingeben: "))
 - exponent = **int**(**input**("Bitte positiven Exponenten eingeben: "))

- Korrigierter Exponent-Rechner:

```
base = int(input("Bitte Basis eingeben: "))
exponent = int(input("Bitte positiven Exponenten eingeben: "))
counter = 1
result = base
while counter < exponent:
    result = result * base
    counter = counter + 1
print(result)
```



- Wir erwarten beim Taschenrechner Zahlen als Eingabe
 - User kann allerdings alles mögliche eingeben...

```
Bitte Basis eingeben: d
```

```
Traceback (most recent call last):
```

```
File "<stdin>", line 1, in <module>
```

```
File "C:\<USER>\test.py", line 7, in <module>
```

```
    base = int(input("Bitte Basis eingeben: "))
```

```
ValueError: invalid literal for int() with base 10: 'd'
```

- Ungültige Eingaben (bzw. dadurch geworfene Fehler) müssen abgefangen werden
 - Über **try** & **except**

```
try:
```

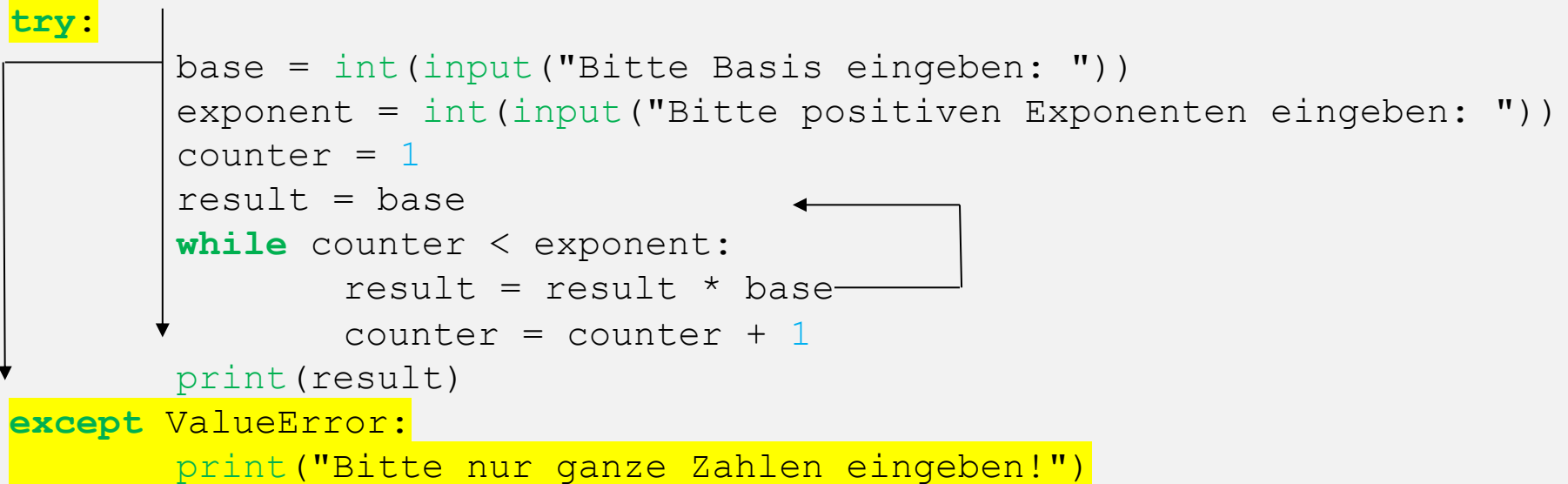
```
    #some code
```

```
except ERROR_NAME:
```

```
    #error handling
```


- (Erneut) korrigierter Exponent-Rechner:

```
try:
    base = int(input("Bitte Basis eingeben: "))
    exponent = int(input("Bitte positiven Exponenten eingeben: "))
    counter = 1
    result = base
    while counter < exponent:
        result = result * base
        counter = counter + 1
    print(result)
except ValueError:
    print("Bitte nur ganze Zahlen eingeben!")
```





- Python bietet eine Vielzahl an möglichen Fehlertypen an:
 - `IndexError`
 - Gegebener Index außerhalb des gültigen Bereichs (z.B. zu groß)
 - `NameError`
 - Name – z.B. einer Variablen oder Funktion – nicht gefunden
 - `SyntaxError`
 - Python-Syntax nicht korrekt
 - `ValueError`
 - Variable hat den richtigen Typ, aber einen ungeeigneten Wert
 - `TypeError`
 - Operation auf den Typ nicht anwendbar. Beispiel: `INTEGER + STRING`
- Liste aller Fehler in der Dokumentation:
 - <https://docs.python.org/3/library/exceptions.html>

Kapitel 5

Bisher: Einfaches Durchlaufen des Codes " von oben nach unten"

- Abgesehen von Schleifen und Fehlern

Entscheidungswege: if / else

Datentyp: None

Schleife: for

- Schleifenanweisungen



- (Nicht-)Ausführung von Codeblöcken abhängig von definierten Bedingungen
- **if** BEDINGUNG:
 - Der folgende Codeblock wird nur ausgeführt, wenn die Bedingung zutrifft
- **else**:
 - Der Codeblock wird nur ausgeführt, wenn das vorherige "if" NICHT zutraf
- **elif** BEDINGUNG:
 - Kombination aus **else** und **if**
- Prüfung auf Gleichheit mit ==
 - Warum nicht = ?



- Beispiel:

```
number = int(input("Bitte Zahl eingeben: "))  
if number > 0:  
    print("Zahl ist positiv")
```



- Beispiel:

```
number = int(input("Bitte Zahl eingeben: "))  
if number > 0:  
    print("Zahl ist positiv")  
else:  
    print("Zahl nicht positiv")
```



- Beispiel:

```
number = int(input("Bitte Zahl eingeben: "))
numberIsPositiv = number > 0
if numberIsPositiv:
    print("Zahl ist positiv")
else:
    print("Zahl nicht positiv")
```



- Beispiel:

```
number = int(input("Bitte Zahl eingeben: "))  
if number > 0:  
    print("Zahl ist positiv")  
elif number < 0:  
    print("Zahl ist negativ")
```




- Beispiel:

```
number = int(input("Bitte Zahl eingeben: "))  
if number > 0:  
    print("Zahl ist positiv")  
elif number < 0:  
    print("Zahl ist negativ")  
else:  
    print("Zahl ist 0")
```

- Objekte können "leer" erstellt werden, bspw.:

```
result = None
```

- Sinnvoll unter anderem zur Übersichtlichkeit
- Beispiel:
 - Try/Except um den gesamten Code
 - Fehler kann aber nur an zwei Stellen auftreten

```
try:
    base = int(input("Bitte Basis eingeben: "))
    exponent = int(input("Bitte positiven Exponenten eingeben: "))
    counter = 1
    result = base
    while counter < exponent:
        result = result * base
        counter = counter + 1
    print(result)
except ValueError:
    print("Bitte nur ganze Zahlen eingeben!")
```

- Mit Verwendung von None

```
base = exponent = None
try:
    base = int(input("Bitte Basis eingeben: "))
    exponent = int(input("Bitte positiven Exponenten eingeben: "))
except ValueError:
    print("Bitte nur ganze Zahlen eingeben!")

if not base == None and not exponent == None:
    counter = 1
    result = base
    while counter < exponent:
        result = result * base
        counter = counter + 1
    print(result)
```



- Negation von Bedingungen: **not** BEDINGUNG
- Manchmal sinnvoller, gegenteilige Fälle abzufangen

```
not a == 0 #alle Fälle außer a gleich 0
```

- Beispiel:

```
dividend = int(input("Bitte Dividend eingeben: "))  
divisor = int(input("Bitte Divisor eingeben: "))  
if not divisor == 0:  
    result = dividend / divisor  
    print(result)
```

- `raise(FEHLER)` wirft (bzw. "verursacht") den angegebenen Fehler

```
try:
    base = int(input("Bitte Basis eingeben: "))
    exponent = int(input("Bitte positiven Exponenten eingeben: "))
    if exponent < 0: #In unserer Rechnung nicht bedacht
        raise(ValueError)
    counter = 1
    result = base
    while counter < exponent:
        result = result * base
        counter = counter + 1
    print(result)
except ValueError:
    print("Bitte nur positive ganze Zahlen eingeben!")
```



- "While"-Schleife sehr beschränkt
- Stumpfes Abfragen einer Bedingung vor jedem Durchlauf
- Iterieren durch Listen eher mühselig
- Beispiel Buchstabenzähler pro Wort:

```
myList = ['IoT', 'ist', 'toll']  
index = 0  
while index < len(myList):  
    word = myList[index]  
    print(word + ":" + str(len(word)))  
    index += 1
```



- Zum Iterieren über Listen besser geeignet: for

```
for x in LISTE:
```

- Beispiel Buchstabenzähler:

```
myList = ['IoT', 'ist', 'toll']  
for word in myList:  
    print(word + ":", str(len(word)))
```

- Definition der Variable "word" wird von Schleife übernommen
- Kombinierbar mit if/else, etc.



- **for** x **in** y nicht auf Listen beschränkt

- Beispiel String:

```
myWord = 'Internet of Things'  
for letter in myWord:  
    print(letter)
```

- Jeder Buchstabe im String wird ausgegeben
- Auch Leerzeichen werden ausgegeben



- **range()** gibt eine Zahlenfolge zurück
 - range(ZAHL): Ganze Zahlen von 0 (inklusive) bis ZAHL (exklusiv)
 - range(ZAHL1, ZAHL2): Ganze Zahlen von ZAHL1 (inklusive) bis ZAHL2 (exklusiv)
 - range(ZAHL1, ZAHL2, ZAHL3,): Ganze Zahlen von ZAHL1 (inklusive) bis ZAHL2 (exklusiv) in ZAHL3-Schritten
- Anzeigen aller geraden Zahlen kleiner als der Input:

```
number = int(input("Bitte Zahl eingeben: "))  
for x in range(number):  
    print(x)  
    if x % 2 == 0:  
        print("gerade")
```



- Weitere Commands in Schleifen:
- **break** beendet die komplette Schleife
- **continue** beendet den aktuellen Schleifendurchlauf

```
lowerNumber = int(input("Bitte kleinere Zahl eingeben: "))
upperNumber = int(input("Bitte größere Zahl eingeben: "))
for num in range(lowerNumber, upperNumber):
    if num < 0:
        print("Bitte nur positive Zahlen eingeben")
        break
    if num % 2 == 0:
        print("Found even number", num)
        continue
    print("Found odd number", num)
```



- **for**-Schleifen kommen mit weiteren eigenen Commands
- **else** wird ausgeführt, wenn kein **break** erreicht wurde
 - Etwas unintuitiv im Vergleich zum **if/else**, hat aber Daseinsberechtigung
- Sinnvolles Beispiel: Primzahlcheck

```
#Algorithmus zum Finden von Primzahlen
for n in range(2, 10):
    for x in range(2, n):
        if n % x == 0:
            print(n, 'equals', x, '*', n//x)
            break
    else:
        # Schleife wurde durchlaufen, ohne dass ein Faktor gefunden wurde
        print(n, 'is a prime number')
```

- Seltene Situation:
 - Syntaktisch eine Anweisung benötigt
 - Programm soll aber nichts machen
 - Bspw. als Platzhalter oder zur Übersichtlichkeit des Codes
- Möglichkeit 1: Sinnloser Befehl
 - Unsauber und unschön zu lesen `dsagdhgdg = 0`
- Möglichkeit 2:
 - Sogenannte "null operation", bei Ausführung geschieht nichts `pass`
- Best Practice: Clean Coding beim Bau von Code-Gerüsten

```
def beispiel_funktion():  
    TODO: hier sollte etwas nützliches passieren  
    pass
```

Kapitel 6 – Code-Style

Gründe für einheitlichen Codestyle

Grund-„Regeln“

"Easier to ask for forgiveness than permission"

"If it looks like a duck and quacks like a duck, it's a duck"

"Explicit is better than implicit"



- Fremden Code lesen kann anstrengend sein
 - Oder eigenen, uralten...
- Deshalb: Einen gewissen Stil einbehalten
- Für Python hat sich Python Enhancement Proposal 8 standardisiert
 - <https://www.python.org/dev/peps/pep-0008/>
- Wesentliche Punkte:
 - Keine Tabs zum Einrücken, sondern 4 Leerzeichen
 - Variablennamen "camelCase" schreiben (`myGeniusVariable`)
 - Funktions- & Methodennamen mit "_" trennen (`calculate_fibonacci()`)
 - Kommentare in eigene Zeilen
 - Benutzung von „# TODO: Das mache ich noch“ für spezielle Hervorhebung in IDE
 - Zeilen nicht ewig lang machen (nicht länger als 79 Zeichen)



- EAFP: "Easier to ask for forgiveness than permission"
 - "Python Coding Style" geht davon aus, dass Elemente / Objekte vorhanden sind
 - Variablen
 - Schlüssel
 - ...
- Einfach ausgedrückt:
 - Wenn die Existenz einer Variable nicht gesichert ist
 - => **try** & **except**
- Ohne EAFP:

```
if not myInsecureVar == None:  
    print(myInsecureVar)  
Else  
    print(errorMsg)
```



- EAFP: "Easier to ask for forgiveness than permission"
 - "Python Coding Style" geht davon aus, dass Elemente / Objekte vorhanden sind
 - Variablen
 - Schlüssel
 - ...
- Einfach ausgedrückt:
 - Wenn die Existenz einer Variable nicht gesichert ist
 - => **try** & **except**
- Mit EAFP:

```
try:  
    print(myInsecureVar)  
except NameError:  
    print(errorMsg)
```

Hier sieht man schon
„explicit is better than implicit“: Nicht
einfach wie wild werfen,
auch explizit fangen!



- "If it looks like a duck and quacks like a duck, it's a duck"
- Es ist viel interessanter, was ein Element/Objekt **kann**, nicht was es **ist**
 - Ähnlich dem EAFP: Python vertraut darauf, dass das Objekt es kann
- Beispiel `VARIABLE [ INDEX ]` :

```
myVar[0]
```
- Wenn `myVar...`
 - Eine Liste ist: Erster Eintrag der Liste als Rückgabe
 - Ein String ist: Erstes Zeichen als Rückgabe ("Erster Eintrag der Zeichenkette")
 - Eine Zahl ist: `TypeError`
- Und hier sieht man EAFP: Try and fail...



- "Explicit is better than implicit"
- Code sollte direkt beim Lesen verständlich sein (hier: Kommentare)

```
number = int(input("Bitte Zahl eingeben: "))

# Even numbers should be divided by 2
if number % 2 == 0:
    number /= 2

# Negative odd numbers get their absolute value
elif number < 0:
    number = - number

# Otherwise, number should remain unchanged
else:
    pass
```



- Erste Aussage in einem Modul, Funktion, etc.
 - **Was** wird gemacht, nicht **wie** wird es gemacht
- Zu verwenden in
 - Modulen
 - Öffentlichen Funktionen von Modulen
 - Öffentlichen Klassen von Modulen
- Syntax: `""" Text. """`
- Beispiel:

```
def function(a, b):  
    """Do X and return a list."""  
    #Code
```

- Wichtig ist die Lesbarkeit, siehe auch <http://www.python.org/dev/peps/pep-0257/>

Kapitel 7 – Datenstrukturen

Garbage Collector

Weiteres zu Listen

Datentyp: Tupel

Datentyp: Set

Datentyp: Dictionary

Schleifentechniken

Operatoren **in** & **is**

Type Checking



- Beim Experimentieren entstehen viele unnötige Variablen
- `del VARIABLE` löscht eine Variable aus dem Speicher
- Meistens nicht benötigt, da Python einen "Garbage Collector" besitzt
 - "Parallel" zu jedem Python-Programm ausgeführt
 - z.B. Löschen von nicht mehr benötigten Variablen aus dem Speicher
- Grundsätzliche Arbeitsweise: es wird gezählt, ob etwas noch gebraucht wird (reference count)
- Further read:
 - <https://rushter.com/blog/python-garbage-collector/>
 - <https://docs.python.org/3/library/gc.html>



- Neben `list` existieren noch weitere Datenstrukturen
- z.B.: Von Python vorgegebene:
 - Tupel
 - Mengen
 - Dictionaries
 - ...
- Unendlich viele weitere Datenstrukturen, die in Bibliotheken definiert werden (Bspw. `numPy`)
- Eigene?

- Nochmal: Listen sind nicht statisch, sondern veränderbar

```
>>> myList = ['IoT', 'ist', 'toll']
```

- Ersetzen eines Elements an Position 2:

```
>>> myList[1] = "war" #['IoT', 'war', 'toll']
```

- Auch Größe der Liste ist variabel:

```
myList.append("gewesen")
```

Hängt an das Ende der Liste an

```
#['IoT', 'war', 'toll', 'gewesen']
```

```
myList.insert(2, "wieder")
```

Fügt das Element an dem Index ein, nachfolgende Elemente werden verschoben

- Automatische Beschränkung des Index auf Listenlänge (1000 würde auch ans Ende anhängen)

```
#['IoT', 'war', 'wieder', 'toll', 'gewesen']
```

```
myList.remove("wieder") #ohne Rückgabe
```

Löscht das erste Element mit dem gegebenen Wert:

```
#['IoT', 'war', 'toll', 'gewesen']
```

Element an Position 3 löschen:

```
myList.pop(3) #das entfernte Element wird zurückgegeben
```

```
#['IoT', 'war', 'toll']
```

- del funktioniert auch hier

```
del myList[1:3]
```

```
#['IoT']
```

„inplace“-Funktionen und
solche mit Rückgabewert

- Weitere Funktionen in der Dokumentation
 - <https://python-reference.readthedocs.io/en/latest/docs/list/>

```
myList.clear()
```

```
#[]
```




- Listen besitzen eine "List Comprehension" Syntax
 - Deklaration versteht Code
 - Listen können direkt bei der Deklaration passend erstellt werden
- Beispiel „Listenerstellung“ **ohne** List Comprehension:

```
>>> a = []
>>> for x in 'abracadabra':
>>>     a.append(x)
>>> a
['a', 'b', 'r', 'a', 'c', 'a', 'd', 'a', 'b', 'r', 'a']
```



- Listen besitzen eine "List Comprehension" Syntax
 - Deklaration versteht Code
 - Listen können direkt bei der Deklaration passend erstellt werden

- Syntax :

```
newList = [expression for item in list]
```

- Beispiel „Listenerstellung“ **mit** List Comprehension:

```
>>> a = [x for x in 'abracadabra']  
>>> a  
['a', 'b', 'r', 'a', 'c', 'a', 'd', 'a', 'b', 'r', 'a']
```

```
>>> a = [2*x for x in 'abracadabra']  
>>> a  
['aa', 'bb', 'rr', 'aa', 'cc', 'aa', ...]
```



- Listen besitzen eine "List Comprehension" Syntax
 - Deklaration versteht Code
- Beispiel Filter:

```
# alle Einträge außer a, b und c
>>> a = [x for x in 'abracadabra' if x not in 'abc']
>>> a
['r', 'd', 'r']
```

```
# sortiere alles aus, was keine Zahl ist und wandel
# den Rest in Integer um
>>> l = [int(x) for x in ['1', 'foo', '2', 'bar'] if x.isdigit()]
>>> l
[1, 2]
```

- Kompaktheit: Viel Logik in wenig Zeilen
- Geschwindigkeit: Schneller als Schleifen durch Listen



- Weiterer Sequenzdatentyp: Tupel
- Mehrere Werte, die durch Komma getrennt sind
- Umfasst von runden Klammern

```
myTupel = 1, 3, "hallo"  
>>>myTupel  
(1, 3, 'hallo')  
  
>>>myTupel[2]  
'hallo'
```



- Strings und Listen haben einige gemeinsame Eigenschaften:
 - Bspw. Slicing oder Indizierung
- Tupel können genau das auch!
 - (warum also....?)
- Tupel sind unveränderlich
 - Anzahl der Elemente unveränderlich
 - Einzelne Elemente nicht veränderbar
 - Wobei... Falls ein Element eines Tupels ein mutable Objekt ist (bspw. Liste), dann kann dieses wohl verändert werden
- Liste: üblicherweise homogene Elemente sowie Zugriff über Iteration
- Tupel: meist heterogene Menge sowie Zugriff über Index oder Unpacking



- "Tuple Packing":

```
t = 123, 456, 789
```

- "Sequence Unpacking":

```
a, b, c = t
```

```
⇒ a = 123
```

```
⇒ b = 456
```

```
⇒ c = 789
```

- Mehrfachzuweisungen (Kapitel 2.2.2) im Grunde genommen nur eine Kombination aus " Tuple Packing" und "Sequence Unpacking"

```
a, b, c = 123, 456, 789
```



- Teilweise ist die Reihenfolge bei Sammlungen egal, vielmehr ist das Vorhandensein wichtiger
 - z.B.: Einkaufsliste
- Python bietet Sets ("Mengen")
 - Ungeordnete Sammlung von Elementen
 - Keine doppelten Einträge
- Syntaktisch sehr ähnlich zu Listen und Tupeln:

```
warenkorb = {'Apfel', 'Orange', 'Birne', 'Orange', 'Banane'}
```

- Sets erkennbar an geschweiften Klammern

```
warenkorb = {'Apfel', 'Orange', 'Birne', 'Orange', 'Banane'}
```

- Ohne Wirkung (aber auch ohne Fehler) hier:

```
warenkorb.add('Orange')
```

- Umwandlung in Set:

```
>>> a = set('abracadabra')  
>>> a  
{ 'a', 'r', 'b', 'c', 'd' }
```

- Set enthält keine doppelten Einträge!
- Umwandlung nur mit iterierbaren Objekten möglich (Strings, Listen, etc.)
 - Nicht z.B. mit Integer



- Auch Sets besitzen eine "Set Comprehension" Syntax
 - Ist der "List Comprehension Syntax" nahezu gleich
 - Geschweifte statt runde Klammern
- Beispiel:

```
# alle einzelnen Einträge
>>> a = {x for x in 'abracadabra'}
>>> a
{'a', 'b', 'r', 'c', 'd',}
```



- Auch Sets besitzen eine "Set Comprehension" Syntax
 - Ist der "List Comprehension Syntax" nahezu gleich
 - Geschweifte statt runde Klammern
- Beispiel Filter:

```
# alle einzelnen Einträge außer a, b und c
>>> a = {x for x in 'abracadabra' if x not in 'abc'}
>>> a
{'r', 'd'}
```

- Sogenannter "Mapping Datatyp"

	Listen / Sequenzen	Tupel	Sets	Dictionaries
Indizierung	Zahlen	Zahlen	Keine	Schlüssel
Ordnung	Über Indizes	Über Indizes	Nein	Nein
Doppelte Einträge	Ja	Ja	Nein	Nein

- Sets augenscheinlich ähnlich zu Dictionaries
 - Keine doppelten Einträge, ungeordnet
- Aber: Schlüssel können aus Strings und Zahlen bestehen
- Syntaktisch dem Set ähnlich
 - Einträge bestehen aus KEY : VALUE

```
>>> d = {'hallo': 'hello', 'tschüss': 'bye'}
```



- Zugriff:

```
>>> d = {'hallo': 'hello', 'tschüss': 'bye'}
>>> d['hallo']
'hello'
```

- Key / Value können beliebige Strukturen sein, z.B.:

```
>>> myTupel1 = "hallo", "english"
>>> myTupel2 = "hallo", "french"
>>> myTupel3 = "hallo", "russian"
>>> myValue1 = "hello"
>>> myValue2 = "bonjour"
>>> myValue3 = "привет"
>>> d = {myTupel1 : myValue1, myTupel2 : myValue2, myTupel3 : myValue3}
>>> d[myTupel1]
'hello'
>>> d[("hallo", "russian")]
'привет'
```



```
>>> tel = {'alice': 4098, 'bob': 4139}
>>> tel['carol'] = 4127 #{'alice': 4098, 'bob': 4139, 'carol': 4127}
>>> tel['alice']
4098

>>> del tel['bob']
#{'alice': 4098, 'carol': 4127}

>>> tel['dave'] = 4127
#{'alice': 4098, 'carol': 4127, 'dave': 4127}

>>> list(tel.keys()) #Liste der Schlüssel als Rückgabe
['alice', 'carol', 'dave']
```



- Ausgabe der Einträge:

```
>>> tel = {'alice': 4098, 'bob': 4139}  
>>> for entry in tel:  
...     print(entry)  
...
```

```
Alice  
bob
```



- Änderungen an der Schleife nötig
 - Genauer: An dem Teil der Sammlung, über die iteriert wird

```
>>> for key in tel.keys():
alice
bob

>>> for value in tel.values():
4098
4139

>>> for entry in tel.items():
('alice', 4098)
('bob', 4139)
```



- Elemente in `items()` sind als Tupel abgebildet

⇒ Key / Value über Index ansprechbar

```
tel = {'alice': 4098, 'bob': 4139}
for entry in tel.items():
    print(entry[0], "hat die Nummer", entry[1])
```

alice hat die Nummer 4098

bob hat die Nummer 4139



- Bei Sequenzen (z.B. Listen) ist manchmal die aktuelle Position interessant
- `enumerate` (SEQUENZ)
 - Erwartet ein iterierbares Objekt
 - Gibt ein Tupel aus Index und Element zurück

```
myList = ['tic', 'tac', 'toe']  
for index, value in enumerate(myList):  
    print(index, value)  
  
0 tic  
1 tac  
2 toe
```

- "index, value" deutet hier auch auf ein Sequence Unpacking hin
- Sehr wichtige Funktion, größerer Code kommt kaum ohne `enumerate()` aus
- Das Konstrukt zählt automatisch mit, kein "i=0" und "i++" notwendig

- Auch iterieren mehrerer Schleifen gleichzeitig möglich
- Befehl: `zip(ITERABLE1, ITERABLE2, ...)`

```
>>> questions = ['Name', 'Auftrag']
>>> answers = ['Lancelot', 'die Suche nach dem Heiligen Gral']
>>> for q, a in zip(questions, answers):
...     print('Was ist dein ' + q + '? Er ist ' + a + '.')
...
Was ist dein Name? Er ist Lancelot.
Was ist dein Auftrag? Er ist die Suche nach dem Heiligen
Gral.
```

- Ebenfalls wichtige Funktion mit häufiger Anwendung

- Besonders für Sammlungen interessant: Test, ob Element in der Sammlung vorhanden ist
- Operator **in** prüft genau das
 - Übrigens: Schreibweise aus der **for**-Schleife ist ein Sonderfall davon

```
>>> basket = {'Apfel', 'Orange', 'Apfel', 'Orange', 'Banane'}  
>>> 'Orange' in basket  
True  
>>> 'Kiwi' in basket  
False
```

```
>>> basket = {'Apfel', 'Orange', 'Apfel', 'Orange', 'Banane'}  
>>> 'orange' in basket  
False  
>>> 'orange' in [x.lower() for x in basket]  
True
```



- Besonders bei "unbekannten" Elementen interessant: Test auf Gleichheit
- Inhaltlich: == Operator

```
>>> a = [1, 2, 3]
>>> b = [1, 2, 3]
>>> a == b
True
```

- Gleicher Inhalt, aber eigentlich verschiedene Objekte

```
>>> id(a), id(b)
(26420688, 26449240)
```



- Besonders bei "unbekannten" Elementen interessant: Test auf Gleichheit
- Tatsächlich gleiches Objekt bzw. gleicher Verweis: **is**

```
>>> a = [1, 2, 3]
>>> b = [1, 2, 3]
>>> a is b
False
>>> id(a), id(b)
(26420688, 26449240)
```



- Besonders bei "unbekannten" Elementen interessant: Test auf Gleichheit
- Beispiel:

```
>>> a = [1, 2, 3]
```

```
>>> b = a
```

```
>>> b is a
```

```
True
```

```
>>> b == a
```

```
True
```

```
>>> b = a[:]
```

```
>>> b is a
```

```
False
```

```
>>> b == a
```

```
True
```



Trotz “Duck Typing” Konvention:

- Möglichkeit des “Type Checkings”

Optionen:

```
>>> type() # Liefert den Typ  
>>> isinstance(obj, type) # Liefert Boolean
```

Beides verwendbar aber der Unterschied liegt im Detail!

Kapitel 8

Lebensdauer von Variablen, Definitionen, etc

Import von Modulen

Eigene Funktionen

- Mit Parameter

Lokale Namen



- Mit Beenden des Interpreters gehen Definitionen verloren
 - Feste Variablenwerte (Konstanten)
 - Funktionen
 - Nach Neustarten müssen diese also neu definiert werden
- ⇒ Auslagerung in eigene sogenannte "Module"



- Erstellen einer .py Datei im aktuellen Ordner
 - z.B.: Sammlung von eigenen Konstanten ("constants.py")

```
defaultTemperature = 21  
bodyTemperature = 36  
name = "Python"
```

- Im Interpreter:

```
# Dateiname ohne Endung  
>>> import constants
```

- Aufruf:

```
>>> constants.defaultTemperature  
21
```

- Beispiel: Beim Coden mit Neuronetzen
 - constants.py auf dem Notebook: debug=true, layer=wenig, zellen=wenig
 - constants.py auf dem Cluster: debug=false, layer=viele, zellen=viele
 - Coding, ohne stets die Einstellungen zu ändern



- Python bietet etliche Standardfunktionen (sogenannte „built-ins“)
- Reicht zum Ausprobieren, aber nicht in der Realität

⇒ Python lässt Definition eigener Funktionen zu

- Ebenfalls in eigenen Modulen auslagerbar
- Syntax:

```
def METHODENNAME () :  
    code
```

```
>>> def print_hallo():  
>>>     print("hallo")  
  
>>> print_hallo()  
hallo
```

- Am Beispiel Fibonacci-Folge bis zur Zahl 3, als Datei calculator.py

```
def fib():  
    max = 3  
    a, b = 0, 1  
    while a < max:  
        print(a, end=' ')  
        a, b = b, a+b  
    print()
```

- Aufruf `calculator.fib()` gibt jetzt die Zahlenfolge 0 1 1 2 aus

```
>>> import calculator
```



- Statische Funktionen sind nur bedingt sinnvoll

```
def fib(max):  
    a, b = 0, 1  
    while a < max:  
        print(a, end=' ')  
        a, b = b, a+b  
    print()
```

- Die Funktion `calculator.fib()` erwartet jetzt einen Parameter, statt der festen 3

```
>>> calculator.fib(15)  
0 1 1 2 3 5 8 13
```



- Häufige Verwendung von Modul-Werten / -Funktionen
 - Einfache Fehlerquelle durch Tippfehler
 - Macht den Code unübersichtlicher
- Bindung an lokale Namen:

```
>>> fib = calculator.fib
>>> fib(15)
0 1 1 2 3 5 8 13
```

Kapitel 9

Beispiel IoT-Sensoren

- Anwendung des bisher Gelernten

Standard-Bibliotheken

Ausgabe-Techniken

Klassen in Python

Vererbungen

Lesen & Schreiben von Dateien



- Beispielszenario: Wir haben eine Menge von IoT-Devices – Thermometer in verschiedenen Räumen – und wollen diese über Python:
 - Steuern (zum Feed hinzufügen und vom Feed trennen)
 - Auslesen
- Definition in eigenem Modul



- Zur Speicherung der aktiven Sensoren:

```
activeSensors = []  
  
def activate(sensorName):  
    activeSensors.append(sensorName)  
    print("Sensor " + sensorName + " aktiviert")  
  
def deactivate(sensorName):  
    activeSensors.remove(sensorName)  
    print("Sensor" + sensorName + " deaktiviert")
```



- Anzeigen aller aktiven Sensoren:

```
def show_active_sensors():  
    for x, sensor in enumerate(activeSensors):  
        print(x, sensor)
```



- Auslesen des Messwertes
- Am besten zufällig in einem Bereich
- Default-mäßig keine Option zur Erstellung zufälliger Zahlen
- Python bietet Reihe an Standard-Bibliotheken
 - Diese müssen allerdings erst importiert werden



- Standard-Bibliothek: sys

```
>>> import sys
```

- Schnittstelle zum Betriebssystem, -befehlen & -konstanten, u.a.:

- sys.getsizeof(object)
 - Rückgabe: Größe des Objekts in Bytes
- sys.getdefaultencoding()
 - Rückgabe: Name des Default-Encodings, meist "UTF-8"
- sys.maxsize
 - Rückgabe: Maximale Größe von Listen, Integers, Strings, etc...
- sys.modules
 - Rückgabe: Liste aller geladenen Module

- <https://docs.python.org/2/library/sys.html>



- Standard-Bibliothek: math

```
>>> import math
```

- Sammlung von mathematischen Befehlen & Konstanten, u.a.:

- math.ceil(x)
 - Rückgabe: Aufgerundeter Wert von x
- math.log(x, base)
 - Rückgabe: Logarithmus von x zur Basis base
- math.pi
 - Rückgabe: π in der verfügbaren Genauigkeit

- <https://docs.python.org/3/library/math.html>



- Standard-Bibliothek: Reguläre Ausdrücke

```
>>> import re
```

- Sammlung zur Verwendung von regulären Ausdrücken, u.a.:

- `re.compile(x)`
 - Rückgabe: Kompilierter regulärer Ausdruck `x`
- `re.findall(pattern, string)`
 - Rückgabe: Liste aller Treffer im String
- `re.match(pattern, string)`
 - Rückgabe: Boolean, ob mindestens ein Treffer vorhanden ist

```
>>> p = re.compile(r'\d+')
>>> p.findall('12 drummers drumming, 11 pipers piping, 10 lords a-leaping')
['12', '11', '10']
```



- Standard-Bibliothek: random

```
>>> import random
```

- Sammlung zur Verwendung von Zufall, u.a.:

- random.choice(sequence)
 - Rückgabe: Ein zufälliges Element aus der Sequenz
- random.random()
 - Rückgabe: Zufälliger Float zwischen 0.0 und 1.0 [exklusiv]

- <https://docs.python.org/3/library/random.html>



- Auslesen des Wertes (hier zufällig zwischen 20-30):

```
import random

def read(sensorName):
    if sensorName in activeSensors:
        temperature = 20 + 10 * random.random()
        return temperature
    else:
        raise ValueError("Kein Sensor mit diesem Namen aktiv.")
```

- return definiert den Rückgabewert einer Funktion
- Random-Modul von Python wird verwendet



- Auslesen des Wertes (hier zufällig zwischen 20-30):

```
import random

def read(sensorName):
    if sensorName in activeSensors:
        temperature = 20 + 10 * random.random()
        return temperature
    else:
        return "Kein Sensor mit diesem Namen aktiv."
```

- return definiert den Rückgabewert einer Funktion
- Random-Modul von Python wird verwendet
- Vorher wurde Fehler geworfen, jetzt wird stets was zurück gegeben
- Best Practice: konsistent bleiben...



- Alle Sensoren ausschalten:

```
def shutdown():  
    activeSensors.clear()
```

- Erinnerung: `clear()` leert die Liste



- Änderung der `activate` Methode
 - Wir haben eine feste Sammlung an verfügbaren Sensoren
 - Anwendungsfall für Sets

```
availableSensors = {"Kitchen", "Kitchen2", "Bathroom", "Basement"}

def activate(sensorName):
    if sensorName in availableSensors:
        activeSensors.append(sensorName)
        print("Sensor " + sensorName + " aktiviert")
    else:
        print("Sensor existiert nicht")
```



- Ausgabe lässt sich formatieren
- Beispiel: Alle aktiven Sensoren auslesen

```
def read_active():  
    for sensor in activeSensors:  
        print("Sensor " + sensor + " zeigt " + str(read(sensor)) + " an.")
```

- String-Konkatenation wird langsam unübersichtlich



- `format()` ersetzt die `{}` markierten Stellen mit dem entsprechenden Parameter
- Wesentlich besser lesbar
- Allgemeine Form:

```
'{0} und {1}'.format(VALUE_1, VALUE_2)
```

- Im Beispiel:

```
def read_active():  
    for sensor in activeSensors:  
        print('Sensor {0} zeigt {1} an.'.format(sensor, read(sensor)))
```



- Ausgabe lässt sich formatieren
- Beispiel: Alle aktiven Sensoren auslesen
- Auch tabellarisch möglich:

```
print('{0:10} {1:10}'.format(sensor, read(sensor)))
```

Ausgabe:

```
kitchen      21.355495879399896  
kitchen2     26.814767843687013
```

- Ein einfaches :ZAHLE gibt die minimale Länge des Feldes an
 - Kürzere Inhalte werden mit Leerzeichen aufgepolstert
- Insbesondere bei Fließkomma-Zahlen jedoch selten einzuhalten



- Neben :ZAHL existieren noch weitere Optionen
- :ZAHL.NACHKOMMASTELLENf dient der Darstellung von solchen Floats
- Beispiel mit zwei Nachkommastellen:

```
print('{0:10} {1:10.2f}'.format(sensor, read(sensor)))
```

Ausgabe:

<i>Kitchen</i>	<i>21.36</i>
<i>Kitchen2</i>	<i>26.81</i>

- Zu kurze Floats werden mit „0“ aufgefüllt



- Anwendungsfall: Wir wollen den Stromverbrauch der Sensoren berechnen und haben einen Maximalwert für den Strom definiert
- Bisher: Sensor "Kitchen" hat keinen festen, zugeordneten Wert für den Stromverbrauch
- Ein Dictionary löst das Problem nicht direkt, da wir verschiedene Küchensensoren mit verschiedenen Werten haben können



- "Einfache" Art: zweite Liste
- Separate Liste enthält Stromverbrauch pro Sensor-Art
 - Aktivieren erhöht den Verbrauch
 - Deaktivieren senkt ihn wieder
- Nachteile:
 - Liste muss zwingend in gleicher Reihenfolge sein
 - Jetzt zwei Listen zu pflegen
 - Übersichtlichkeit?



- "Schöne" Art: Objekte / Klassen
- Python kann eigene Klassen erstellen, aus denen Objekte erstellt werden können
- Auf deutsch:
 - In der Natur gibt es u.a. die Pflanzenart Baum
 - Baum besitzt einige Eigenschaften, wie Fotosynthese betreiben oder Anzahl Zweige
 - Jede Baumart (Birke, Eiche, Kirschbaum, etc.) ist ein Baum
 - Jede Baumart hat die typischen Eigenschaften eines Baumes



- "Schöne" Art: Objekte / Klassen
- Python kann eigene Klassen erstellen, aus denen Objekte erstellt werden können
- In Python:
 - Python besitzt u.a. die Klasse `String` (also Zeichenketten)
 - `String` bietet eigene Methoden, wie `format()` oder `len()` sowie Eigenschaften und Konstanten
 - Jede erstellte Zeichenkette ist ein Objekt dieser Klasse
 - Jede Zeichenkette kann auf die Funktionen der Klasse zugreifen



```
class Square:
    length = 5
    def get_area(self):
        return self.length * self.length
```

- `self` referenziert hierbei immer auf das aufrufende Objekt
 - Vergleichbar z.B. mit `this` in Java
 - Wird immer als erster Parameter mitgegeben
 - beim Aufruf "unsichtbar", wird nicht mitgetippt



```
class Square:
    length = 5
    def get_area(self):
        return self.length * self.length
```

- Verwendung:

```
>>> mySquare = iot.Square()
>>> mySquare.length
5
>>> mySquare.get_area()
25
```



- Variablen und Funktionen besitzen gewisse "Gültigkeitsbereiche"

```
class Square:
    length = 5
    def get_area(self):
        area = self.length * self.length
        return area
```

Ordneten Bereichen

- length kann objekt-weit – deshalb auch in der Funktion – verwendet werden
 - Für getArea() gilt dasselbe
- area allerdings nur innerhalb der Funktion, kein Zugriff von außen

```
>>> mySquare = iot.Square()
>>> mySquare.length
5
>>> mySquare.area
AttributeError: 'Square' object has no attribute 'area'
```



- Gültigkeitsbereiche lassen sich einschränken
 - `__NAME` Variable/Funktion nur noch innerhalb der Klasse verwendbar

```
class Square:
    __length = 5

    def get_length(self):
        return self.__length

    def get_area(self):
        area = self.__length * self.__length
        return area
```

```
>>> mySquare = iot.Square()
>>> mySquare.length
AttributeError: 'Square' object has no attribute 'length'
```



- Zurück zum Sensor-Problem
- Neue Klasse: Sensor
 - Eigenschaften: Name, Stromverbrauch
 - Funktionen: Auslesen



- Neue Klasse: Sensor

```
class Sensor:

    def __init__(self, name, consumption):
        self.name = name
        self.consumption = consumption

    def read(self):
        temperature = 20 + 10 * random.random()
        return temperature
```

- Feldernamen (name, consumption) müssen nicht vorher deklariert werden



- Neue Klasse: Sensor

```
def __init__(self, name, consumption):  
    self.name = name  
    self.consumption = consumption
```

- Spezielle Methode
- Definiert wie das Objekt erstellt wird
 - Hier mit zwei weiteren Parametern: Name und Consumption (`self` wird immer übergeben)
 - Werte werden in diesem Beispiel in die Eigenschaften des Objekts geschrieben

```
>>> mySensor = iot.Sensor("kitchen", 3)
```

- Anpassungen an bestehende Methoden **des Moduls** nötig
- Keine `availableSensors`-Liste mehr nötig, da wir mit der Erstellung der Objekte "arbeiten" können (`availableSensors` schon)
- Kleinere Sachen wie `.name`:

```
def activate(sensor):  
    activeSensors.append(sensor)  
    print("Sensor " + sensor.name + " aktiviert")
```

- Verwendung der `read`-Methode der Sensoren durch `read`-Methode des Moduls:

```
def read(sensor):  
    if sensor in activeSensors:  
        temperature = sensor.read()  
        return temperature  
    else:  
        return "Kein Sensor mit diesem Namen aktiv."
```

- Berechnung des Stromverbrauchs (innerhalb des Moduls):

```
def calculate_consumption():  
    consumption = 0  
    for sensor in activeSensors:  
        consumption = consumption + sensor.consumption  
    return consumption
```

Neue Klassen erben fast immer von 'object'

Also bei „class Klasse():“

Vererbung auch von bestehenden Klassen möglich

```
class TempSensor(Sensor):  
  
    def __init__(self, consumption):  
        name = 'TemperaturSensor'  
        super().__init__(name, consumption)  
  
    def read_in_kelvin(self):  
        temperature = 20 + 10 * random.random() + 273.15  
        return temperature
```



Objekt `t` hat nun eine Funktion `“read_in_kelvin()”` und die Funktion der Elternklasse `“read()”`

```
>>> t = TempSensor()  
>>> t.read_in_kelvin()  
  
>>> t.read()
```



Instanziierung:

```
class Vehicle(object):  
    pass  
  
class Truck(Vehicle):  
    pass
```

Verhalten und Unterschied:

```
>>> isinstance(s, iot.Sensor)           # returns True  
>>> type(s) == iot.Sensor               # returns True  
  
>>> isinstance(ts, iot.Sensor)          # returns True  
>>> type(ts) == iot.Sensor              # returns False
```



- Speichern von Messwerten in .csv Dateien wünschenswert
- Python Modul: csv `import csv`
- Datei öffnen, umständlich:

```
file = open(FILEPATH+NAME)
data = file.read()
print(data)
file.close() #Datei schließen nach Benutzung!
```




- Speichern von Messwerten in .csv Dateien wünschenswert
- Python Modul: csv `import csv`
- Datei öffnen, einfacher:

```
with open(FILEPATH+NAME) as myFile:  
    data = myFile.read()
```
- `with` übernimmt einiges an Syntax
 - z.B. das Schließen der Datei
 - Außerdem besseres Fehler-Handling



- open kann mehr Argumente annehmen, bspw.:
 - Zweites Argument: Lese- / Schreibmodus ("r" / "w")
 - `newline` = Bestimmung des Zeilenende-Zeichens
- Schreiben einer Datei im selben Ordner, Einträge durch Leerzeichen getrennt:

```
import csv

def write_temperatures():
    with open('mappe.csv', 'w', newline='') as csvfile:
        writer = csv.writer(csvfile)
        for sensor in activeSensors:
            writer.writerow([sensor.name] + [sensor.read()])
```



- Auslesen von Werten aus .csv Datei
 - Dem Schreiben sehr ähnlich
- Datei im Beispiel im selben Ordner wie ausgeführtes Skript

```
with open('mappe.csv', newline='') as csvfile:  
    reader = csv.reader(csvfile)  
    for row in reader:  
        print(', '.join(row))
```

Kapitel 10 – Fortgeschrittene Themen

Lambda Funktionen

- Was sind Lambda Funktionen?
- Wofür werden sie gebraucht?

Threading



- Lambda Funktionen stammen aus der funktionalen Programmierung
- Python unterstützt den Lambda-Operator
- Dient zur Erzeugung anonymer Funktionen
 - Klein, häufig Einzeiler
 - Ohne Namen
- "Wegwerf-Funktionen"
 - **Eigentlich** keine Wiederverwendung im späteren Code
 - In Variablen schreibbar
- Häufig im Bereich map/filter/reduce verwendet



- Allgemeiner Aufbau:

```
lambda ARGUMENTLISTE: AUSDRUCK
```

- Einfachstes Beispiel:

```
>>> f = lambda x, y : x + y  
>>> f(3, 5)  
8
```



- `map(func, seq)` erwartet eine Funktion und eine Sequenz als Parameter
 - Alle Werte der Sequenz werden der Funktion unterzogen
- Sinnvolleres Beispiel: Umwandlung Celsius in Fahrenheit
 - $4.5 * \text{Celsius} + 32 = \text{Fahrenheit}$

```
>>> celsius = [39.2, 36.5, 37.3, 37.8]
>>> fahrenheit = list(map(lambda x: (float(9)/5)*x + 32, celsius))
>>> print(fahrenheit)
[102.56, 97.700000000000003, 99.140000000000001, 100.03999999999999]

>>> c = list(map(lambda x: (float(5)/9)*(x-32), fahrenheit))
>>> print(c)
[39.200000000000003, 36.5, 37.300000000000004, 37.799999999999997]
```



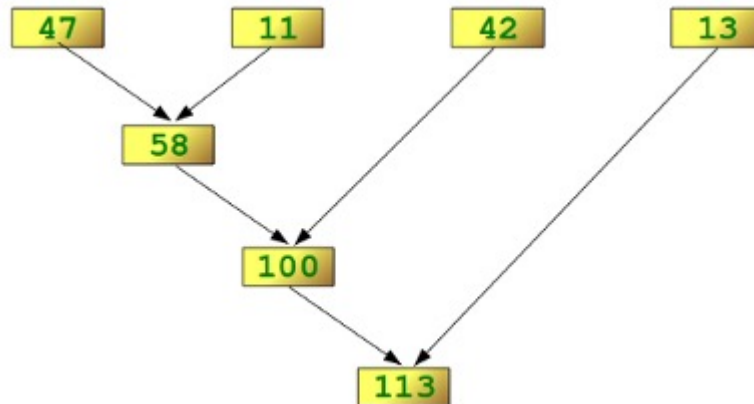
- `filter(func, seq)` erwartet eine Funktion und eine Sequenz als Parameter
 - Funktion `func` muss einen Boolean zurückgeben
 - Nur Werte, für die die Funktion `True` ergibt, werden behalten
- Beispiel: Filtern von geraden / ungeraden Zahlen

```
>>> fib = [0,1,1,2,3,5,8,13,21,34,55]
>>> result = list(filter(lambda x: x % 2, fib))
>>> print(result)
[1, 1, 3, 5, 13, 21, 55]
>>> result = list(filter(lambda x: x % 2 == 0, fib))
>>> print(result)
[0, 2, 8, 34]
```


- `reduce(func, seq)` erwartet eine Funktion und eine Sequenz als Parameter
 - Werte der Sequenz werden paarweise der Funktion unterworfen

```
>>> reduce(lambda x,y: x+y, [47,11,42,13])  
113
```

- Schematisch dargestellt:



From functools import reduce



- Umfangreiche Berechnungen meist zeitintensiv
- Bisher
 - Programm in der Zeit nicht benutzbar
 - Nur eine Berechnung gleichzeitig
- Parallelisierung mit threading!



- Vor Python 3:

```
import thread
```

- Seit Python 3 obsolet, import deshalb mit _ (Rückwärtskompatibilität)
 - Nun: module threading

```
import _thread
```

- Zum Starten eines Threads:

```
_thread.start_new_thread(function, (args[, args2, args3,...]))
```



- Beispiel Wurzelberechnung:

```
import _thread
import math

def calc_root(a):
    result = math.sqrt(a)
    print(a, result)
    return result

_thread.start_new_thread(calc_root, (99,))
_thread.start_new_thread(calc_root, (999,))
_thread.start_new_thread(calc_root, (1733,))
```

- (99,) für tuple, (99) würde int ergeben
- Bewirkt nichts, da mit Beenden des Hauptprogrammes alle Threads geschlossen werden!

- „Hacky“ Lösung: Warten auf Eingabe

```
import _thread
import math

def calc_root(a):
    result = math.sqrt(a)
    print (a, result)
    return result

_thread.start_new_thread(calc_root, (99,))
_thread.start_new_thread(calc_root, (999,))
_thread.start_new_thread(calc_root, (1733,))

c = input("Eingabe.")
```

- Idee: Solange warten, bis kein Thread mehr aktiv ist

```
import _thread
import math

numThreads = 0
def calc_root(a):
    global numThreads
    numThreads += 1
    result = math.sqrt(a)
    print (a, result)
    numThreads -= 1
    return result

_thread.start_new_thread(calc_root, (99,))
_thread.start_new_thread(calc_root, (999,))
_thread.start_new_thread(calc_root, (1733,))

while numThreads > 0:
    pass
```



- Idee: Solange warten, bis kein Thread mehr aktiv ist
- Scheitert ebenfalls an der Realität
- Schleife wird erreicht, bevor ein Thread den Zähler erhöht
- Außerdem entspricht

```
num_threads += 1
```

```
num_threads = num_threads + 1
```
- Worst-Case Szenario:
 - Thread 1: Liest 0 aus Variable
 - Thread 2: Liest 0 aus Variable
 - Thread 1: Erhöht 0 um 1 => 1
 - Thread 2: Erhöht 0 um 1 => 1



- Atomare Operationen werden immer in einem Stück durchgeführt („unteilbar“)
- `num_threads += 1` besteht aus zwei atomaren Operationen!
- Python bietet „Locks“ zur Definition kritischer Bereiche
 - Solche Bereiche werden nicht aufgesplittet

- Mit Locks und „Gestartet“-Überprüfung:

```
import _thread
import math

numThreads = 0
threadStarted = False
lock = _thread.allocate_lock()
def calc_root(a):
    global numThreads, threadStarted
    lock.acquire()
    threadStarted = True
    numThreads += 1
    lock.release()
    result = math.sqrt(a)
    print(a, result)
    lock.acquire()
    numThreads -= 1
    lock.release()
    return result

_thread.start_new_thread(calc_root, (99,))
_thread.start_new_thread(calc_root, (999,))
_thread.start_new_thread(calc_root, (1733,))

while not threadStarted:
    pass

while numThreads > 0:
    pass
```

Kapitel 11 – The Zen of Python



```
>>>import this
```

The Zen of Python, by Tim Peters

Beautiful is better than ugly.
Explicit is better than implicit.
Simple is better than complex.
Complex is better than complicated.
Flat is better than nested.
Sparse is better than dense.
Readability counts.
Special cases aren't special enough to break the rules.
Although practicality beats purity.
Errors should never pass silently.
Unless explicitly silenced.
In the face of ambiguity, refuse the temptation to guess.
There should be one-- and preferably only one --obvious way to do it.
Although that way may not be obvious at first unless you're Dutch.
Now is better than never.
Although never is often better than *right* now.
If the implementation is hard to explain, it's a bad idea.
If the implementation is easy to explain, it may be a good idea.
Namespaces are one honking great idea -- let's do more of those!